An Approximate Digital Compute-In-Memory Macro with Reconfigurable Computational Precision for Neural Network Acceleration

Heng You[1], Zixiao Zhan[2], Yan Gao[2], Guanghua Zhao[3], Yumei Zhou[3], Shushan Qiao[3*], Jia Yuan[1*]
[1]Nanjing Institute of Intelligent Technology, Nanjing, China
[2]Nanjing University of Information Science and Technology, Nanjing, China
[3]Institute of Microelectronics of Chinese Academy of Sciences, Beijing, China
[*]Email: qiaoshushan@ime.ac.cn, yuanj@niit.ac.cn

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562521

***Abstract*—Among various Compute-in-Memory (CIM) architectures, approximate digital CIM (APPRO-DCIM) offers a promising balance among robustness, computational accuracy, and area/power overhead, showing considerable application potential. However, current APPRO-DCIM macros typically support only a single precision level, making it difficult to achieve an optimal trade-off between computational accuracy and efficiency in complex application scenarios. To address this issue, this paper presents an APPRO-DCIM macro with reconfigurable computational precision. The circuit adopts a NAND–NOR interleaved approximate logic structure, combined with a customized logic rearrangement and reconfiguration mechanism, enabling the support of multiple precision levels within a single macro. Fabricated in a 55 nm process, the macro has a storage capacity of 16Kb and occupies an area of 0.083 mm². The APPRO-DCIM macro supports three distinct precision modes. Measurement results demonstrate that in low-precision mode, the macro delivers a throughput of 3.2768TOPS and an energy efficiency of 5624TOPS/W; in high-precision mode, it achieves 1.6384TOPS and 2699TOPS/W; and in full-precision mode, it achieves 0.8192TOPS and 1315TOPS/W.**

***Keywords*—Compute-in-Memory, approximate computing, reconfigurable computational precision, logic rearrangement, neural network computations**

I. Introduction

With the rapid advancement of artificial intelligence (AI), the efficient storage and processing of massive amounts of data have become one of the major challenges for AI chips [1]. In the conventional von Neumann architecture, the physical separation of memory and processing units necessitates continuous data movement across bandwidth-limited buses. This continuous data movement incurs significant energy overhead, while the limited bandwidth of the buses concurrently constrains processing speed and overall system performance. To overcome these limitations, compute-in-memory (CIM) architecture has rapidly evolved in recent years, emerging as an effective solution to address these bottlenecks.

Among various types of memories, static random-access memory (SRAM) offers distinct advantages, including low operating voltage, fast read/write speeds, high endurance, low power consumption, and excellent compatibility with standard CMOS processes. In recent years, research on energy-efficient, SRAM-based CIM has attracted widespread interest. Depending on the computational approach, SRAM-based CIM can be classified into analog [2-7] and digital implementations [8-13]. Analog CIM performs multiply-accumulate (MAC) operations using analog quantities such as current or charge. However, this approach is sensitive to process, voltage, and temperature (PVT) variations, leading to reduced system accuracy and robustness. In contrast, digital CIM employs digital logic circuits to execute MAC operations, effectively addressing the computational accuracy and robustness issues of analog CIM macros. However, the large digital adder tree incurs significant area and power consumption overhead.

To achieve an optimal balance between computational accuracy and efficiency, approximate digital compute-in-memory (APPRO-DCIM) [14-16] has emerged in recent years. By introducing approximate computation into digital CIM macros, this approach significantly reduces the area and power overhead introduced by adder trees while maintaining robustness against PVT variations. Furthermore, unlike the unpredictable errors of analog CIM implementations, computational errors in APPRO-DCIM are deterministic. This predictability allows formal mathematical models of these approximations to be developed and integrated into neural network training, thereby effectively mitigating the impact of approximation errors on recognition accuracy and ultimately achieving an improved balance between computational efficiency and accuracy.

Approximate computing logic was first applied to digital CIM in [14], which employed an interleaved AND/OR gate structure to replace portions of the adder tree, achieving significant reductions in area and power overhead. Furthermore, through an approximation-aware training method, this design further mitigates the impact of approximate computation on neural network recognition accuracy, preliminarily demonstrating the superiority of APPRO-DCIM. In [16], an OR-gate-based approximate computation logic was adopted. By replacing the first level of the adder tree with OR gates, area and power consumption of the CIM macro were reduced. To further suppress computational errors, the design proposed an in-memory swapping mechanism to detect and swap error-prone activation inputs and their corresponding weights. Similar approaches were applied to RRAM-based digital CIM [17], which similarly employed OR-gate-based approximation logic while constraining at least one weight in adjacent channels to zero to prevent all OR gate inputs from being "1". In [15], a hybrid approximation scheme was proposed, designing two types of approximate logic based on XOR and OR gates for the high-bit and low-bit positions in multi-bit MAC operations, respectively. This was combined with block-level pruning during training to reduce the occurrence probability of critical error patterns.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:45 UTC from IEEE Xplore. Restrictions apply.

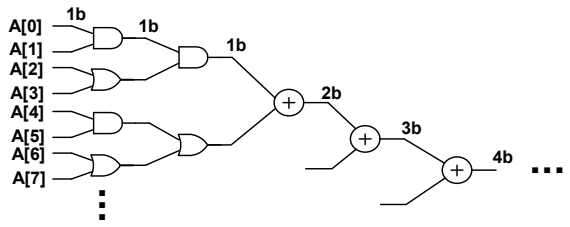


Fig.1. Schematic of the fundamental approximation logic.

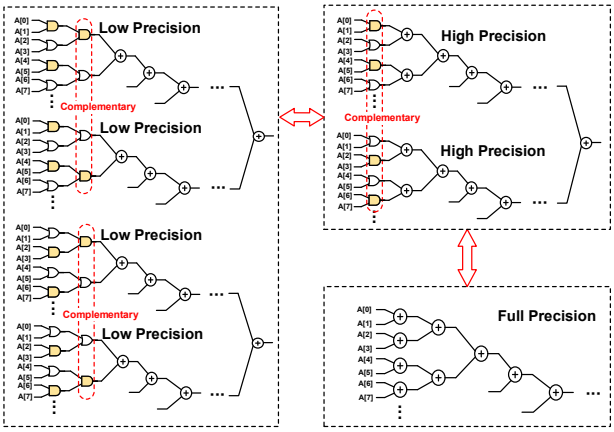


Fig.2. Reconfigurable-precision approximate-summation strategy.

Despite these improvements in area and energy efficiency, the approximation precision of these CIM macros is fixed. In practical applications, different tasks demand varying levels of computational precision. Even within the same neural network, different layers require distinct optimal precision levels. Consequently, fixed approximation precision struggles to maximize area and energy efficiency. To address this issue, this paper proposes an APPRO-DCIM macro with reconfigurable computational precision. The proposed design employs NAND/NOR approximate logic and reorganizes the order of approximate computing units across adjacent rows. By leveraging the complementary error characteristics of NAND and NOR gates, this rearrangement enables error compensation between neighboring rows during summation operations. When higher computational precision is required, adder trees from adjacent rows can be applied to the same MAC operation, and their outputs can be accumulated—thereby achieving high-precision computation using low-precision logic elements.

II. The Proposed APPRO-DCIM

A. Approximate Computing Logic Analysis

In digital CIM designs, the adder tree typically dominates both area and power consumption. To reduce this overhead, approximation mechanisms can simplify the adder tree logic by replacing precise full adders with simpler circuits. The fundamental approximation logic employed in this paper is illustrated in Fig.1, which replaces conventional adders with alternating AND/OR gates. When adjacent inputs are 00 or 11, using AND logic for approximation introduces no computational error, nor does using OR logic. When adjacent inputs are 01 or 10, using AND logic introduces a −1 computational error, while OR logic introduces a +1 error. By interleaving AND and OR logic, these errors partially cancel each other out, thereby reducing the overall computational error caused by approximation.

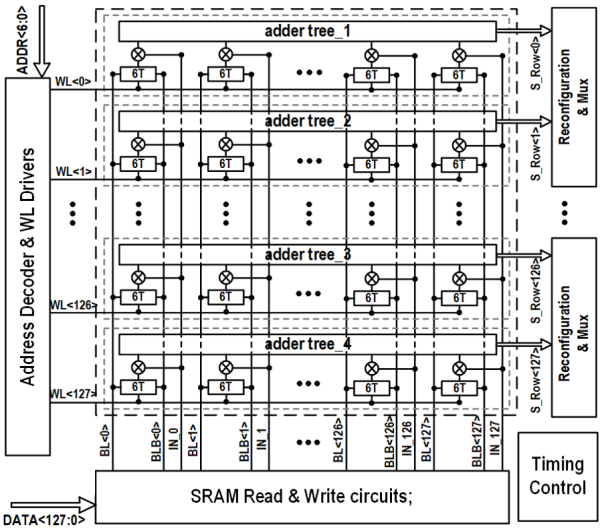


Fig.3. Architecture of the proposed APPRO-DCIM.

To enable APPRO-DCIM to adapt to diverse application scenarios, this design proposes a precision-reconfigurable scheme within the approximate summation logic, allowing different computational precision levels to be achieved through architectural reconfiguration. Analysis of the error distributions in AND and OR logic reveals that when the same data is processed by both types of gates, their computational errors are complementary. Averaging the two outputs thus produces a high-precision result. Based on this insight, this paper proposes a reconfigurable-precision approximate-summation strategy. As shown in Fig.2, A[1]-A[7] represent partial products obtained from bitwise multiplication of two sets of 1-bit data. During partial product summation, two layers of full adders are replaced with simple AND/OR gates. To achieve reconfigurable computational precision, the approximation logic of adjacent rows is rearranged. This ensures that the summation results from adjacent approximation units exhibit complementary error distributions. For low-precision computation, each row performs calculations independently. For high-precision computation, adjacent rows process the same data set. Since the computational errors of adjacent rows are complementary, averaging their results yields high-precision results.

B. Architecture of the Proposed APPRO-DCIM

The architecture of the proposed APPRO-DCIM is illustrated in Fig.3. This APPRO-DCIM primarily consists of a storage/computing array, read/write peripheral circuits, and reconfigurable summation circuits. Since the macro employs a fully digital approach, no charge/discharge operations are required for BL/BLB during computation. Consequently, 6T SRAM cells are utilized to reduce area overhead. To enable reconfigurable computational precision, each row of the integrated adder tree in this design exhibits distinct variations. Organized into groups of four rows, the AND and OR logic gates within the approximation logic are arranged differently across these rows. By combining the computational logic of multiple rows, varying levels of computational precision can be achieved. The peripheral circuit, similar to that of conventional SRAM, handles functions such as input latching and address decoding. The summation circuit is responsible for merging partial results from multiple rows to deliver the desired computational precision.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:45 UTC from IEEE Xplore. Restrictions apply.

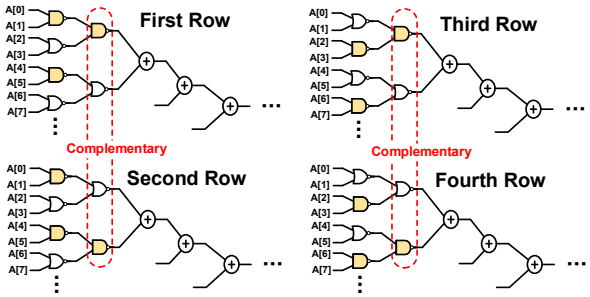


Fig.4. Summation logic diagram for four adjacent rows.

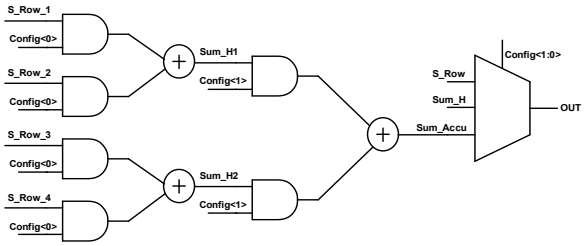


Fig.5. Schematic of the reconfigurable summation circuit.

Fig.4 illustrates the summation logic diagram for four adjacent rows. To reduce the area and power consumption of the summation logic, NAND and NOR gates are used to replace the AND and OR gates in Fig.1. As shown in Fig.4, the second-level summation logic for rows 1 and 2 is complementary, while their first-level summation logic is identical. By reorganizing the summation logic for rows 1 and 2, the two sets of low-precision computation logic can be reconfigured into a single high-precision computation logic. Similarly, Rows 3 and 4 also feature complementary second-level logic and identical first-level logic, enabling their recombination into another set of high-precision logic. Furthermore, since the high-precision logic formed by recombining Rows 1/2 and that formed by Rows 3/4 exhibit complementary approximation characteristics, combining the results of these two groups yields a full-precision computational outcome.

Fig.5 shows the schematic of the reconfigurable summation circuit. As shown in Fig.5, the proposed APPRO-DCIM achieves flexible configuration between different computational precisions through a set of configuration signals. When Config is 00, it indicates a low-precision approximate summation operation is being performed, requiring no logic restructuring. In this case, two sets of AND logic isolate the summation result of each row from the adders, and the multiplexer (MUX) directly outputs the result of each row. When Config is 01, it indicates high-precision summation operation. This requires reorganizing the logic for every two rows: the first AND logic group is left open, while the second is cut off. The first-stage summing adder accumulates the sum results from every two rows to obtain the high-precision sum. When Config is 11, indicating full-precision summation operation, the logic for every four rows is reorganized. Both sets of AND gates remain open. The first-stage adder sums the results from every two rows, and the second adder combines these two high-precision results to produce the full-precision calculation result.

During low-precision computation, the macro stores one set of weight data per row, with inputs provided externally. Each row independently completes one set of MAC operations per clock cycle. At this time, the reconfigurable

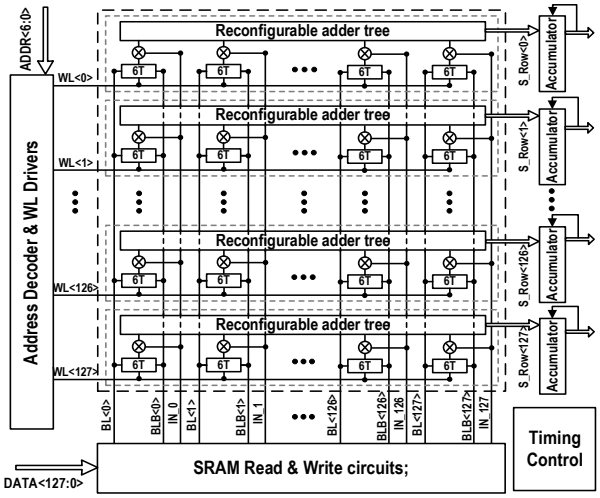


Fig.6. Architecture of multi-cycle multiplexing reconfigurable DCIM.

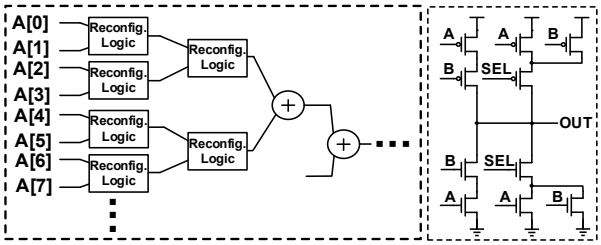


Fig.7. Schematic of the configurable approximate adder tree and the corresponding reconfigurable logic.

summation logic is disabled via control signals. During high-precision computation, two rows store one set of weight data. The computational resources for two rows are used to simultaneously perform a set of MAC operations. The results from both rows are accumulated by the summation logic to yield the final result. Similar to low-precision computation, high-precision computation also completes one calculation per clock cycle. For full-precision computation, one set of weights is stored per four rows. The computational resources of these four rows are utilized to perform the MAC operation on one set of data. The results from these four rows are then reorganized into a full-precision result. Similarly, during full-precision computation, one set of calculations is still completed per clock cycle.

C. Multi-Cycle Multiplexing Extension

The above design achieves reconfigurable computational precision through spatial reconfiguration of computing logic. This approach can also be extended to a multi-cycle multiplexing architecture. By designing configurable approximate logic combined with different computational cycle control, different computational precision levels can be realized. Fig.6 shows the architecture of multi-cycle multiplexing reconfigurable APPRO-DCIM. Unlike the adder tree in Fig.3, which incorporates distinct approximate logic combinations, the multi-cycle multiplexing architecture integrates logic-reconfigurable adder trees in every row, allowing the approximate logic to be configured as either NAND or NOR gates through configuration signals. Fig.7 shows the schematic of the configurable approximate adder tree and its corresponding reconfigurable logic. As shown in Fig.7, when the configuration signal SEL is low, the computation logic functions as a NAND gate; when SEL is high, it operates as a NOR gate.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:45 UTC from IEEE Xplore. Restrictions apply.

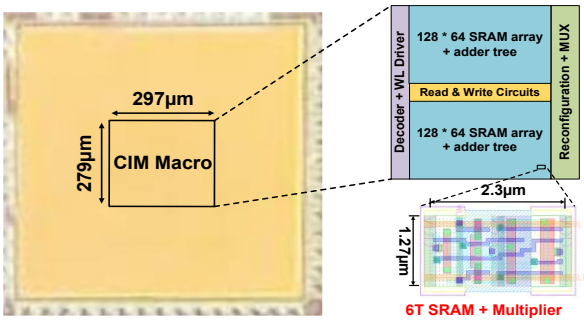


Fig.8. Micrograph of the test-chip.

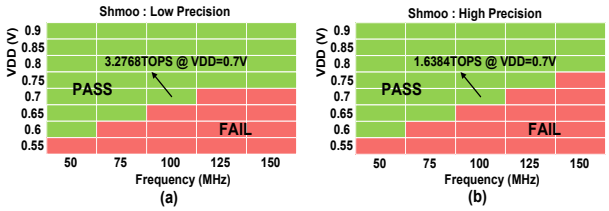


Fig.9. Shmoo plot of the test-chip in different precision levels.

The cycle-multiplexed CIM operates as follows: In low-precision mode, both approximation stages use a NAND-NOR alternating pattern. Each row completes a set of calculations per cycle without iterative accumulation. In high-precision mode, each computation spans two cycles. The second-stage approximation logic alternates between NAND-NOR and NOR-NAND configurations across cycles to ensure complementary errors, while the first stage maintains a fixed NAND-NOR pattern. The two cycle results are accumulated to produce the high-precision output. In full-precision mode, each computation extends over four cycles. During the first two cycles, the first stage uses a NAND-NOR pattern while the second stage toggles between NAND-NOR and NOR-NAND. In the latter two cycles, the first stage switches to NOR-NAND while the second stage continues its alternation. All four results are accumulated to generate the full-precision output.

Compared with the spatially-reconfigurable CIM, the cycle-multiplexed CIM employs relatively more complex approximation logic, resulting in some area overhead (<20%). But the cycle-multiplexed CIM stores one set of weights per row across different configurations, offering higher memory cell utilization during high-precision computations. In practical scenarios, specific implementation approaches can be flexibly selected based on actual requirements.

III. Measurement Results

To validate the effectiveness of the proposed APPRO-DCIM, a test chip was fabricated using a 55 nm CMOS process. Fig.8 shows the micrograph of the test chip along with the layout of the proposed CIM macro. As shown in Fig.8, the CIM provides a memory capacity of 16Kb and occupies a core area of 0.083 mm².

Fig.9 presents the shmoo plot of the proposed APPRO-DCIM. As shown in Fig.9(a), when the supply voltage is 0.7V and the clock frequency is 100MHz, the peak throughput of the CIM in low-precision mode reaches 3.2768 TOPS. Similarly, under the same voltage and frequency condition, Fig.9(b) indicates that the macro delivers a peak throughput of 1.6384 TOPS in high-precision mode.

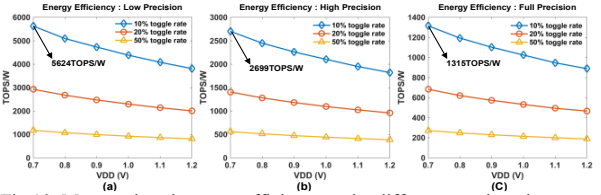


Fig.10. Measured peak energy efficiency under different supply voltages and input toggle rates.

TABLE I Comparison with Previous Works

	JSSC'20 [2]	SSCL'22 [3]	ISCAS'25 [7]	ISSCC'22 [14]	This Work
Technology	65nm	28nm	65nm	28nm	55nm
Type	Analog	Analog	Analog	Digital	Digital
Array Size	16Kb	16Kb	80Kb	16Kb	16Kb
Supply Voltage	0.8	0.7-1	1.2	0.45-1.1	0.7-1.2
Area (mm²)	0.081	0.027	0.076	0.033	0.083
Input/Weight	1b/1b	1b/1b	1b/1b	1b/1b	1b/1b
Reconfigurable Precision	No	No	No	No	Yes
Energy Efficiency (TOPS/W)	671	1607-3182	1305	2219	5624
Throughput (GOPS)	1638	716	NA	9175	3276.8

Fig.10 shows the measured peak energy efficiency of the proposed CIM under different supply voltages and input toggle rates (50% 1s in the array). As shown in Fig.10, at a supply voltage of 0.7 V and an input toggle rate of 10%, the CIM macro achieves a peak energy efficiency of 5624 TOPS/W in low-precision mode, 2699 TOPS/W in high-precision mode, and 1315 TOPS/W in full-precision mode. It can be observed that the energy efficiency of high-precision mode is slightly less than half of that in low-precision mode. This results from two factors: first, the high-precision summation logic is constructed by reorganizing the low-precision logic from two rows; second, additional summation circuit is required during the reorganization process.

To further validate the effectiveness of the proposed CIM macro in neural network applications, LeNet-5 model is applied to MNIST dataset for recognition. In low-precision mode without error-aware training, the recognition accuracy is only 78.07% (baseline is 99.21%). After applying error-aware training, the accuracy improves to 97.87%. When one layer is replaced with high-precision computation, accuracy increases to 98.16%. When all layers are configured to high-precision mode, the recognition accuracy is further enhanced to 98.77%.

IV. Conclusion

In this work, a reconfigurable-precision APPRO-DCIM macro is proposed, which effectively addresses the limitation of fixed computational precision in conventional APPRO-DCIM macros. By employing a NAND-NOR interleaved approximate logic structure with a customized logic rearrangement and reconfiguration mechanism, the proposed design achieves three distinct precision modes within a single macro. The proposed APPRO-DCIM enables dynamic precision-adaptation in CIM systems, providing an optimal balance between computational accuracy and efficiency for varying application requirements. Furthermore, since the proposed CIM macro is implemented using digital logic, it can be easily scaled to advanced process nodes such as 22nm or even 6nm.

Acknowledgment

This work was supported by STI 2030-Major Projects under Grant 2021ZD0201204.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 30,2026 at 12:14:45 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-In-memory for AI edge devices," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 68, no. 5, pp. 1773–1786, May 2021.

[2] Z. Jiang, S. Yin, J. -S. Seo and M. Seok, "C3SRAM: An In-Memory-Computing SRAM Macro Based on Robust Capacitive Coupling Computing Mechanism," IEEE Journal of Solid-State Circuits, vol. 55, no. 7, pp. 1888-1897, July 2020.

[3] H. Oh et al., "Energy-Efficient In-Memory Binary Neural Network Accelerator Design Based on 8T2C SRAM Cell," IEEE Solid-State Circuits Letters, vol. 5, pp. 70–73, 2022.

[4] Z. Yue, Y. Wang, Y. Qin, L. Liu, S. Wei, and S. Yin, "BR-CIM: An efficient binary representation computation-in-memory design," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 69, no. 10, pp. 3940–3953, Oct. 2022.

[5] J. Song, X. Tang, X. Qiao, Y. Wang, R. Wang, and R. Huang, "A 28 nm 16 kb bit-scalable charge-domain transpose 6T SRAM in-memory computing macro," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 70, no. 5, pp. 1835–1845, May 2023.

[6] R. Sehgal et al., "A bit-serial, compute-in-SRAM design featuring hybrid-integrating ADCs and input dependent binary scaled precharge eliminating DACs for energy-efficient DNN inference," IEEE Journal of Solid-State Circuits, vol. 58, no. 7, pp. 2109–2124, Jul. 2023.

[7] T. Liao et al., "Kung-Fu: An Energy-Efficient Compute-In-Memory Approach for Neural Network Inference Using Multi-Level Binary Computing Fusion," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1-5.

[8] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," IEEE Journal of Solid-State Circuits, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[9] Y. -D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm^2 All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications," 2021 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 252-254.

[10] H. Mori et al., "A 4nm 6163-TOPS/W/b 4790−TOPS/mm^2/b SRAM Based Digital-Computing-in-Memory Macro Supporting Bit-Width Flexibility and Simultaneous MAC and Weight Update," 2023 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2023, pp. 132-134.

[11] H. Kim, J. Mu, C. Yu, T. T. Kim, and B. Kim, "A 1–16b reconfigurable 80Kb 7T SRAM-based digital near-memory computing macro for processing neural networks," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 70, no. 4, pp. 1580–1590, Apr. 2023.

[12] H. You, W. Li, D. Shang, Y. Zhou and S. Qiao, "A 1–8b Reconfigurable Digital SRAM Compute-in-Memory Macro for Processing Neural Networks," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 71, no. 4, pp. 1602-1614, April 2024.

[13] J. Hu, W. L. Goh and Y. Gao, "A Digital Compute-in-Memory Macro Featuring Two's Complement Multiplication for LSTM-based Biomedical Signal Classification," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1-5.

[14] D. Wang, C. -T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy and M. Seok, "DIMC: 2219TOPS/W 2569F^2/b Digital In-Memory Computing Macro in 28nm Based on Approximate Arithmetic Hardware," 2022 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2022, pp. 266-268.

[15] C. He et al., "LSAC: A Low-Power Adder Tree for Digital Computing-in-Memory by Sparsity and Approximate Circuits Co-Design," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 71, no. 2, pp. 852-856, Feb. 2024.

[16] M. -G. Lin, J. -P. Wang, C. -Y. Chang and A. -Y. A. Wu, "Approximate Adder Tree Design with Sparsity-Aware Encoding and In-Memory Swapping for SRAM-based Digital Compute-In-Memory Macros," 2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS), Abu Dhabi, United Arab Emirates, 2024, pp. 362-366.

[17] Y. He et al., "An RRAM-Based Digital Computing-in-Memory Macro With Dynamic Voltage Sense Amplifier and Sparse-Aware Approximate Adder Tree," IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 70, no. 2, pp. 416-420, Feb. 2023.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:14:45 UTC from IEEE Xplore. Restrictions apply.